Method Report: Archetype-Aware Card Synergy Analysis

MTGNET

March 6, 2026

Purpose

This pipeline is designed to answer two practical questions about a limited format. First, which groups of cards define real deck archetypes? Second, inside each archetype, which cards are intrinsically strong and which cards mostly become good when they are paired with the right support shell?

The implementation uses only repository evidence: the 17Lands game export, the 17Lands draft export, and card metadata from `cards.csv`. In the current Tarkir run the code learns from 126,902 unique deck builds, 552,428 game rows, and the draft pick export that was supplied alongside them.

1 Input Representation

The code uses three complementary views of the format.

- The game export provides a full `deck_*` representation for each build and a `drawn_*` representation for each individual game. The deck view is used to learn archetypes; the drawn view is used to measure in-game synergy.
- The draft export is grouped by `draft_id`. Picked cards are accumulated with `pick_maindeck_rate` as a soft deck-construction weight. Those draft vectors are then projected into the learned archetype space to provide corroborating draft-side support for card pairs and triplets.
- `cards.csv` contributes card color identity, rarity, mana value, and type information. Color identity is used in the visualization and the rarity/type fields are used to prevent lands and low-frequency rares from dominating the signpost logic.

2 Learning Deck Archetypes

The pipeline first collapses the game file to one row per unique build using the key `(draft_id, build_index)`. This yields a sparse deck matrix

$$X \in \mathbb{R}^{D \times C},$$

where each row is a deck build and each column is a card.

Cards that are too rare to stabilize a cluster or so ubiquitous that they carry almost no archetype signal are filtered before clustering. In the current implementation the clustering feature set keeps cards that appear in at least a minimum number of builds and in at most a fixed share of all builds. This is the first guardrail against the "commons are everywhere" problem.

The remaining matrix is weighted with a TF-IDF style transform,

$$w_{dc} = (1 + \log x_{dc}) \cdot \log\left(\frac{1 + D}{1 + \mathrm{df}_c}\right),$$

so cards that appear in nearly every deck are downweighted while cards that are concentrated into specific shells become more informative. The weighted matrix is then reduced with truncated SVD and clustered with MiniBatch K-means. The output is a hard archetype assignment for every deck build.

This is more transparent than a direct pair-only graph because the archetype is learned from the entire deck, not from whichever two cards happen to be drawn together most often.

3 Signpost Cards

Once deck clusters are available, each card is scored against the rest of the format with a smoothed log-odds comparison. For a cluster k, if a_c is the number of decks in the cluster containing card c and b_c is the number of decks outside the cluster containing the same card, the signpost score is based on

$$\log \frac{a_c + \alpha_c}{N_k - a_c + \beta_c} - \log \frac{b_c + \alpha_c}{N_{\neg k} - b_c + \beta_c},$$

with a prior centered on the card's global prevalence. The prior is mildly rarity-adjusted so that a rare card is not automatically treated as a defining signpost just because it is scarce.

This score is not used in isolation. Lands are removed from the signpost list, and signpost cards are later combined with game-side performance and motif neighbors so that the final network focuses on cards that both define the archetype and matter when games are actually played.

4 Within-Archetype Game Synergy

After clustering, the code makes a second pass over the game export and keeps statistics only inside each learned archetype. The baseline for an archetype is its own game win rate, not the format-wide win rate. This matters because a pair that is average in Boros may be excellent in Abzan, and vice versa.

For each card c in a cluster the code computes a standalone lift,

$$\Delta_{\text{standalone}}(c) = \mathrm{WR}(c \text{ drawn}) - \mathrm{WR}(\text{cluster}).$$

This is the main "good alone" signal.

For each pair (a, b) the code compares the observed pair win rate with an additive log-odds expectation,

$$\mathrm{logit}(\widehat{\mathrm{WR}}_{ab}) = \mathrm{logit}(\mathrm{WR}_a) + \mathrm{logit}(\mathrm{WR}_b) - \mathrm{logit}(\mathrm{WR}_{\text{cluster}}).$$

The pair synergy score is then

$$\Delta_{ab}^{(2)} = \text{logit}(\text{WR}_{ab}) - \text{logit}(\widehat{\text{WR}}_{ab}).$$

Positive values indicate that the pair wins more often together than would be expected from the two single-card win rates.

Triplets are handled analogously. For a triplet (a, b, c) the expected log-odds subtracts the cluster baseline twice,

$$\text{logit}(\widehat{\text{WR}}_{abc}) = \text{logit}(\text{WR}_a) + \text{logit}(\text{WR}_b) + \text{logit}(\text{WR}_c) - 2\,\text{logit}(\text{WR}_{\text{cluster}}).$$

The triplet excess above that expectation is used both as a reportable motif and as a bonus term for the edges that participate in that motif.

5 Combining Deck, Game, and Draft Evidence

A pair is only truly compelling when it is visible in more than one slice of the data. The exported network therefore attaches three different kinds of support to every edge.

- Deck-level pair lift: how often the two cards co-occur in the same learned deck cluster relative to independence.
- Game-level pair synergy: how much the observed win rate of the drawn pair exceeds the additive expectation inside that cluster.
- Draft corroboration: how often the same pair or triplet is assembled in drafts that project to the same archetype.

The edge quality score is a multiplicative combination of those terms with support scaling. The exact constants are heuristic, but the components are interpretable and are exposed in the JSON/HTML outputs.

6 Card Roles

Each node is classified from two axes.

- **Good Alone**: strong standalone lift, small dependence on pair/triplet support.
- **Needs Support**: low standalone lift but high dependency score, meaning the card mostly becomes good inside a reinforced shell.
- **Engine**: both individually strong and heavily tied to pair/triplet structure.
- **Balanced**: everything else.

This role split is what turns the network from a pure co-occurrence picture into an answer to the question "which cards require other cards to be good, and which cards are good on their own?"

7 Visualization Semantics

The HTML output is deliberately archetype-centric. Each segment corresponds to one learned deck cluster. Node fill follows color identity exactly: mono-color cards use their WUBRG RGB color, two-color cards use a split circle, and cards with three or more colors are rendered gold. Edges are brighter and thicker when the within-archetype pair quality is high. Background panels indicate graph communities found on the positive edge subgraph, which helps small "knots" of mutually reinforcing cards stand out visually.

Card hover in the HTML fetches the card image directly from Scryfall by exact card name, so the user can inspect the actual card without bundling a local image library into the repository.

8 Interpretation Notes

The method is intentionally evidence-driven, but it still has limitations.

- Archetypes are hard clusters. Hybrid decks are forced into their nearest cluster even when they share structure with two shells.
- Drawn-game synergy is descriptive, not causal. A positive edge means the pair overperformed when both were seen, not that one card is guaranteed to cause the other to improve.
- Very high-end rares can still dominate the top of the bomb list if enough support exists. The shrinkage and support filters reduce that effect, but they do not remove it completely.

In exchange, the pipeline produces a much more useful description of a draft format than a global pair table: it finds deck families first, signposts them with card-level evidence, and only then asks how pairs and triplets change win rate within each family.